Multilingual RAG Data Store

1. Introduction

1.1 Purpose

This document outlines the high-level architecture and system design for a Basic Retrieval-Augmented Generation (RAG) Data Store designed for document import and semantic search using a PostgreSQL Vector Database.

1.2 Business Use Case

Many organizations store diverse documents like HR policies, project documents, Standard Operating Procedures (SOPs), and technical guides. Employees often struggle to find relevant information quickly. This system acts as a backend component to store, search, and retrieve documents based on semantic meaning rather than just exact keywords. This foundation can later integrate into a full RAG-based AI assistant.

2. Component Design

2.1 Database Layer (Vector Store)

- The system will utilize PostgreSQL with the pgvector extension to store document chunks and embeddings.
- **Table Name:** documents.
- **Schema Fields:** id, document_name, chunk_text, embedding, created_at.

2.2 Document Ingestion Module

- The minimum supported import format is .txt, with optional support for .pdf and .docx files.
- The import pipeline will read the document, split it into smaller text chunks, generate embeddings for each chunk, and store both the chunk and embedding in the PostgreSQL database.

2.3 Multilingual Embedding Module

- The system shall generate vector embeddings for each text chunk.
- To support both Malayalam and English, the architecture must deviate from the originally suggested English-only model (sentence-transformers/all-MiniLM-L6-v2).
- A multilingual model (e.g., via the OpenAI Embeddings API or a multilingual Sentence Transformer variant) will be implemented to handle cross-lingual embedding.

2.4 Semantic Search Engine

- The system will provide a simple search function where users can enter queries.

- The pipeline will convert the query into an embedding, search the PostgreSQL vector database, and return the top 3 or 5 matching chunks.
- The output will display the document name alongside the matching text.

3. Technology Stack

Area	Technology
Language	Python
Database	PostgreSQL
Vector Extension	pgvector
Embedding Model	Multilingual Sentence Transformers or OpenAI API
API Framework	FastAPI (Optional)
UI	simple HTML page
Version Control	GitHub

4. Minimum Deliverables

The project submission will include:

- A GitHub repository containing the codebase.
- Setup instructions detailed in a README.md file.
- A PostgreSQL table creation script.
- Python scripts for both document import and search functionalities.
- Sample testing documents (in English and Malayalam) and screenshots demonstrating import and search outputs.

5. Acceptance Criteria (Test Cases)

- **Infrastructure Setup:** PostgreSQL and the pgvector extension must be installed and enabled successfully, with the table storing names, text, and embeddings.
- **Document Import:** The system must successfully import single and multiple documents, splitting them and storing all chunks correctly.
- **Search Execution:** The system must return the top 5 results sorted by similarity for exact keywords and meaning-based queries, even if exact words differ.
- **Error Handling:** Empty queries must trigger a proper validation message, and invalid files must yield an error without crashing the system.
- **Portability:** The project must run successfully on a different machine by following the README.md setup instructions.

6. Optional Extensions

If time permits, the following features may be added:

- A simple FastAPI endpoint formatted as /search?q=
- PDF document import capabilities.
- A basic web UI.
- Display of the calculated similarity score alongside each search result.
- Administrative options to delete or re-import documents.